API MI CIUDAD · CATÁLOGO TURÍSTICO

Reemplazo de fotografías de lugares

Qué se agregó, cómo se utiliza y cómo se verificó el nuevo endpoint administrativo.

Guía técnica para el equipo

Proyecto api-mi-ciudad · Colón, Entre Ríos

1. El problema y la solución

Antes del cambio, la API permitía subir una imagen con `POST /archivos`, pero no asociarla luego con un lugar turístico existente. El catálogo expone las imágenes de cada lugar, pero no tenía una operación de edición para modificarlas.

Se agregó `PATCH /lugares/:id/imagenes`. Solo lo puede ejecutar una cuenta con rol `admin`. El endpoint reemplaza la lista completa de imágenes y acepta únicamente referencias a imágenes previamente cargadas a esta API.

> **Alcance deliberadamente acotado:** no habilita la edición del nombre, descripción, dirección ni otros datos del lugar. Tampoco cambia ni sube automáticamente las imágenes existentes.

2. Flujo de uso

1. Iniciar sesión con una cuenta administradora y obtener el token.
2. Subir cada fotografía mediante `POST /archivos`.
3. Copiar la URL que devuelve la API.
4. Enviar la lista de URLs a `PATCH /lugares/:id/imagenes`.
5. Leer el lugar actualizado de la respuesta o volver a consultar el catálogo.

Ejemplo de subida de una foto desde una terminal con cURL:

```
curl -X POST "http://localhost:3000/archivos" ^
  -H "Authorization: Bearer <TOKEN_ADMIN>" ^
  -F "archivo=@foto-colon.jpg"
```

La respuesta contiene una URL que se usará en el paso siguiente:

```
{
  "datos": {
    "url": "http://localhost:3000/archivos/imagenes/2026/10/abc123.jpg",
    "tipo": "imagen",
    "bytes": 184512
  },
  "meta": null
}
```

Ejemplo para reemplazar las fotos del lugar `lug-004`:

```
curl -X PATCH "http://localhost:3000/lugares/lug-004/imagenes" ^
  -H "Authorization: Bearer <TOKEN_ADMIN>" ^
  -H "Content-Type: application/json" ^
  -d "{\"imagenes\":[\"http://localhost:3000/archivos/imagenes/2026/10/abc123.jpg\"]}"
```

En PowerShell puede ser necesario reemplazar `curl` por `curl.exe`. Sustituir la URL base, el token y el identificador del lugar por los valores del entorno.

3. Contrato del endpoint

Elemento	Comportamiento
Método y ruta	`PATCH /lugares/:id/imagenes`
Autorización	Token válido de una cuenta con rol `admin`.
Cuerpo	`{ "imagenes": ["URL", "..."] }`; hasta 20 URLs.
Éxito	HTTP 200 con el lugar actualizado en `datos` y `meta: null`.
Lista vacía	`{ "imagenes": [] }` quita todas las fotos.
Lista completa	La lista enviada reemplaza la anterior; no se agregan elementos automáticamente.

Errores esperables

HTTP	Código	Cuándo ocurre
400	`VALIDACION_FALLIDA`	El cuerpo no contiene una lista válida o excede el máximo.
401	`NO_AUTENTICADO`	Falta el token o no es válido.
403	`SIN_PERMISO`	La cuenta existe, pero no tiene rol administrador.
404	`LUGAR_NO_ENCONTRADO`	No existe el lugar solicitado.
422	`ARCHIVO_NO_SUBIDO`	Una URL no corresponde a una imagen existente subida a esta API.

4. Qué hace internamente

- El guard de autenticación comprueba que el rol guardado en la base sea `admin`.
- El DTO valida que `imagenes` sea una lista de cadenas de hasta 20 elementos.
- El servicio comprueba primero que el lugar exista.
- Cada URL se convierte a una clave de archivo y se verifica en el almacenamiento, en la carpeta de imágenes. URLs ajenas, archivos que no existen y archivos que no son imágenes se rechazan.
- La base almacena las claves internas, no las URLs completas. El mapeador genera las URLs públicas como en el resto de la API.
- Se actualiza `actualizadoEn`, incluso si varias operaciones ocurren dentro del mismo segundo, para que la sincronización incremental pueda detectar el cambio.
- Al validar todas las referencias antes de escribir, una URL inválida no deja reemplazada parcialmente la lista.

5. Archivos del proyecto modificados

Archivo	Responsabilidad

`src/modules/places/places.controller.ts`	Declara la ruta, el rol requerido y la documentación Swagger.
`src/modules/places/dto/update-place-images.dto.ts`	Valida la lista de imágenes y su máximo de 20.
`src/modules/places/places.service.ts`	Verifica lugar y archivos, guarda las claves y actualiza la fecha de modificación.
`test/e2e/place-images.e2e-spec.ts`	Pruebas de reemplazo, eliminación, permisos, URLs, límites y sincronización.
`docs/SPECIFICATION.md`	Contrato del endpoint y alcance de edición del catálogo.
`docs/DECISIONS.md`	Decisión provisional D10: permitir a admins cambiar solo las fotos.
`README.md`	Guía de uso y tabla automática de endpoints generada desde Swagger.

6. Pruebas ejecutadas

- Chequeo de tipos TypeScript: correcto.
- Oxlint sobre el módulo de lugares y la nueva prueba: correcto.
- Pruebas e2e nuevas y regresión del catálogo: **90 pruebas correctas**.
- Prueba que comprueba la tabla de endpoints del README: correcta.

Para validar se utilizó Node 24, que cumple el requisito del proyecto. La instalación local del driver SQLite necesitó reconstruirse antes de ejecutar las pruebas; no se modificaron las dependencias declaradas del proyecto.

7. Qué necesita el equipo para publicar fotos

1. Las fotografías deben ser JPG, PNG o WEBP y pesar hasta 5 MB, según las reglas de `POST /archivos`.
2. Debe existir una cuenta administradora habilitada en el entorno de la API.
3. Subir las imágenes, guardar las URLs devueltas y llamar al nuevo endpoint.
4. La aplicación cliente puede refrescar el lugar inmediatamente o sincronizarlo con `actualizadoDesde`.

Una carpeta compartida en el repositorio sigue siendo útil si el grupo solo quiere distribuir imágenes locales dentro de su app. Pero, si quieren que la fotografía nueva forme parte del catálogo servido por la API y aparezca a todos sus clientes, deben usar el flujo de subida y reemplazo descrito aquí.

Documento preparado a partir de los cambios del proyecto api-mi-ciudad.